

QUEUE QNO:5

QUESTION:

Question description

Sathya is an DS expert training youngsters struggling in DS to make them better.

Sathya usually gives interesting problems to the youngsters to make them love the DS.

One such day Sathya provided to the youngsters to solve the task such that, insert an element in a Queue in FIFO order

Youngsters were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

every lines indicates the enqueue of each elements

last line indicates the fine queued elements.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 5 5 6 3 1 2 EXPECTED OUTPUT Enqueueing 5 5 Enqueueing 6 5 6 Enqueueing 3 5 6 3 Enqueueing 1 5 6 3 1 Enqueueing 2 5 6 3 1 2	INPUT (STDIN) 8 1 8 6 4 9 5 3 5 EXPECTED OUTPUT Enqueueing 1 1 Enqueueing 8 1 8 Enqueueing 6 1 8 6 Enqueueing 4 1 8 6 4 Enqueueing 9 1 8 6 4 9 Enqueueing 5 1 8 6 4 9 5 Enqueueing 3 1 8 6 4 9 5 3 Enqueueing 5 1 8 6 4 9 5 3 5

Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD void enqueue(int data)	KEYWORD for(i=front;i<=rear;i++)	KEYWORD enqueue(data);
CYCLOMATIC COMPLEXITY 3	TOKEN COUNT 200	NLOC 44

Source Code:

```
#include <stdio.h>
#define MAX 100
int queue[MAX];
int front = -1;
int rear = -1;
void enqueue(int data) {
    if (rear == MAX - 1) {
        return;
    }
    if (front == -1) {
        front = 0;
    }
    rear++;
    queue[rear] = data;
}

void disp() {
    if (front == -1) {
        return;
    }
    int i;
    for(i=front;i<=rear;i++) {
        printf("%d", queue[i]);
        if (i < rear) {
            printf(" ");
        }
    }
}

int main() {
    int size;
    if (scanf("%d", &size) != 1) return 0;
    for (int j = 0; j < size; j++) {
        int data;
        scanf("%d", &data);
        if (j == 0) {
            printf("Enqueuing %d\n", data);
        } else {
            disp();
            printf(" Enqueuing %d\n", data);
        }

        enqueue(data);
    }
    disp();
    printf("\n");
    return 0;
}
```

Output:

```
5
5 6 3 1 2
Enqueueing 5
5 Enqueueing 6
5 6 Enqueueing 3
5 6 3 Enqueueing 1
5 6 3 1 Enqueueing 2
5 6 3 1 2
```

```
8
1 8 6 4 9 5 3 5
Enqueueing 1
1 Enqueueing 8
1 8 Enqueueing 6
1 8 6 Enqueueing 4
1 8 6 4 Enqueueing 9
1 8 6 4 9 Enqueueing 5
1 8 6 4 9 5 Enqueueing 3
1 8 6 4 9 5 3 Enqueueing 5
1 8 6 4 9 5 3 5
```